# GBR Fix

Project handbook – how the program works, state after version 2.7

A Windows tray program that corrects Optitex 25/26 cut files (GBR) for the cutter, puts back notches that Optitex leaves out (GBR and HP-GL/HPG), and checks the result.

Document date: 13 October 2026
Program version described: **2.7**
Repository: `nggkroze/teokroze`, folder `GbrFix/`, branch `claude/gbr-fix-notch-correction-uub3kq`
Language of the program window and messages: Bulgarian. This handbook is in English; Bulgarian UI texts are given with a translation.

**Status in one paragraph.** The program was developed and tested on Linux (unit tests, headless-browser checks of the window, and your sample files), and compiled for Windows. **It has never been run on Windows, and none of its output has been cut on the machine.** Section 10 lists exactly what is verified and what is not. Test every new output on a trial marker before real cutting.

# 1. What the program is for

Optitex 25 and 26 write cut files that the cutter (through ViewCut) handles wrongly: notches are cut in the wrong way, some are not cut at all. GBR Fix watches the folders where Optitex Marker saves cut files and, for every new or changed file, writes a corrected copy next to it. **The original is never changed.**

| Input | What GBR Fix does |
|---|---|
| `.GBR` (ISO/GBR cut file) | Converts Optitex notch forms that the cutter misreads into the old form it cuts correctly ( `A*M19*B*M15*A*M14` ), removes duplicates, closes U notches, checks the result, and optionally adds notches that Optitex left out (taken from the marker). |
| `.HPG` / `.HPGL` (HP-GL) | Finds the notches that Optitex 26 left out (the last notch of every piece) and writes them into a copy, in the same T form as the other notches of that file. |
| `.MRK` / `.PDS` | Never changed. Read only: the `.MRK` contains a plain-text drawing with every notch and its coordinates; the `.PDS` gives the expected notch counts. Both can be opened in the built-in preview. |

# 2. Background – what is wrong in the Optitex files

## 2.1 GBR files (findings handed over with version 2.0; checked by the conversions against sample files)

- The notch **positions** are correct; the **order and form** of the commands changed between versions.
- **Optitex 15–21** wrote each notch inside the contour: edge point A, `M19` (notch command), point B 3 mm in, `M15` (knife up), back to A. This is the form the cutter handles.
- **Optitex 25 / 26.0** moved all notches to the start of each piece, in front of the contour ( `A*M14*M19*B*M15` …).
- **26.0.4.2 / 26.1** draw the T notch as real cut lines (a slit plus a 3 mm crossbar); with "M19 for cut notches" unticked there is no notch command at all. I notches are written in the old form but 4 mm deep. V, box and U notches are written as cut shapes; the U notch does not return to the edge and trims a sliver off the piece.
- **ViewCut rule** (found by reading how ViewCut parses the file): after a notch end point the *next* move is executed as a cut even if the file says knife up ( `M15` ), unless the file goes straight back to the notch's edge point. Those moves are the "red lines"; the program calls them **phantom cuts**.
- 26.1 also drops some notches from the GBR (30 of 45 in three test exports: one on each of pr, dj and kolan, per set).

## 2.2 HP-GL files (our own finding, 12–13 October 2026)

Sample marker `CH-CHINO-Atatex-PR-LICE-32-44` , plotted with HP-GL, *Metric 1/40* (1 unit = 0.025 mm). Each notch is drawn as a T: `PD E; PD I; PU C1; PD C2; PU E` (edge point E, 4 mm in to I, 4 mm crossbar C1–C2, back to E). The marker has **232** notches; the HPG has **204**. **The last notch of every piece that has notches is missing** – 28 of 28 pieces, one each. The marker ( `.MRK` ) and pattern ( `.PDS` ) are correct; all notches are the same type ( `T_NOTCH` , cut, 8 mm). The

full description for Optitex (English and Bulgarian) is in `GbrFix/docs/Optitex_notch_report_EN.md` and `_BG.md`.

In the GBR of the same marker the individually cut pieces have all their notches. Eight pieces (38 notches) are inside four rectangular outlines and are not written as pieces in the GBR; the HPG draws them individually. Whether this is intended (buffer/block cutting) is an open question to Optitex.

## 3. How a file is processed

1. **Watching.** Windows reports changes (`ReadDirectoryChanges`) in the watched folders and all their sub-folders. Events for `.gbr`, `.hpg`, `.hpgl`, `.mrk`, `.pds` are used.
2. **Settling.** A cut file is processed 2 s after its last event; size and time are checked again after 0.7 s (a file still being written is postponed). Files over 64 MB are skipped. The program's own outputs (suffix `_fixed`, `_fixed_restored`) and folders in the exclude list are ignored. Each size+time is handled once. If the file cannot be read, 20 retries 3 s apart.
3. **Type.** HP-GL is recognised by its content (`PD` / `PU` commands, no `*N`); anything else with `*N` and `M1…` is treated as GBR.
4. **GBR:** `Convert()` repairs the notch forms (3.1); then the marker is looked up (section 4) and, if one is found, `RestoreMissing()` adds missing notches (3.2); `Simulate()` re-reads the result like ViewCut would and counts phantom cuts; the notch count is compared with the marker/pattern.
5. **HPG:** the marker is looked up; `RestoreHPG()` adds missing T notches (3.3).
6. **Output** is written atomically (temporary file, then rename), a report is saved, an entry is added to the "Recent files" list, and a notification is shown (batched).

### 3.1 GBR repairs (`convert.go`, `shapes.go`)

| Found in the file | Result |
|---|---|
| Notches in front of the contour (Optitex 25/26.0) | Moved into the contour at their edge point, as `A*M19*B*M15*A*M14`. This matches the HTML converter v16 byte for byte on the first sample. |
| Exact duplicates; near-duplicates (same A and B within `neardup` units, default 3 = 0.3 mm) | Removed. |
| T notch drawn as cut lines without `M19` (26.1), and the SP4 T-slit form | Turned into an I notch with `M19` at the same point and depth. A geometry check makes sure ordinary contour cuts are never touched. |
| V, box and U notches drawn as shapes | Left as shapes by default. Each kind can be turned into an I notch (settings). A U notch that does not return to the edge is closed to the edge (`close_broken_u`, on by default). |
| Notches damaged by old converters (`A*B*M15*A*M14`) | Optional recovery (`recover_broken_i`, off by default; only for old damaged files). |
| Damaged header (no `ZX` marker length, or text re-saved by an editor/e-mail) | Warning "export the file again from Marker". |

Coordinates are in 0.1 mm. Files are handled as raw bytes so the cp1251 header text is kept unchanged.

### 3.2 Restoring missing notches in a GBR (`mrk.go`)

The marker drawing gives every notch as a line A→B. A marker notch counts as already present if a GBR notch (M19 or shape) lies within 2 mm of A. Otherwise A is placed on the nearest contour segment (squared tolerance `segment_tol` = 400, i.e. 2 mm) and an I notch `A*M19*B*M15*A` is inserted. Notches that do not lie on any contour in the GBR are counted as "unplaced". A notch at the very start of a contour (written before `M14`) is recognised as present – before 2.7 it was added a second time and produced a red cut.

| `restore_mode` | Output |
|---|---|
| `separate` (default) | `NAME_fixed.GBR` only converted; `NAME_fixed_restored.GBR` = same plus the restored notches. Cut the second one after checking it. |
| `same` | One file: `NAME_fixed.GBR` with the restored notches. |
| `off` | Nothing is added; only a warning "N notches missing". |

With `out_mode=folder` the files go into `FOLDER_fixed\NAME.GBR` and `FOLDER_fixed_restored\NAME.GBR` instead.

### 3.3 HP-GL restoring (`hpg.go`, `hpgapp.go`)

1. The file is split into commands (`PU`, `PD`, `SP` ...). Notches are recognised on each pen as `E; PD I; PU C1; PD C2; PU E` (T) or `E; PD I; PU E` (I).
2. **Fit.** Scale (4, 1, 10, 2 or 3.937 units per 0.1 mm), axis swap and cut pen are chosen so that most file notches lie on marker notches (±1 mm). If less than half match, the file "does not fit the marker": nothing is changed and a warning is shown.
3. **Missing** = marker notches without a file notch at the same place *and pointing into the same side* (two touching pieces can have notches 1 mm apart, pointing in opposite directions).
4. **Insert.** Each missing notch goes on the nearest cut segment (1.5 mm tolerance; within 0.5 mm of a corner it is placed on the corner), in the form the file already uses: T if most of its notches are T, with the median slit and crossbar length (4 + 4 mm in the sample).
5. **Only the new lines are added** (`PD E` if needed, `PD I`, `PU C1`, `PD C2`, `PU E`); everything else, including line endings and the end of the file, stays byte for byte. A second run adds nothing.

Output: `NAME_fixed.HPG` (or in a `_fixed` folder). On the sample, 28 notches are added and the second pass finds 232 of 232.

## 4. Finding the marker (and pattern)

Marker usually saves `.MRK` / `.PDS` in a different folder than the exported cut file, so the marker is looked up in this order (`markers.go`, `FindMarker`):

| # | Source | Accepted when |
|---|---|---|
| 1 | The marker the user chose for this file (**"Посочи .MRK..."** = "Choose .MRK..."), remembered in `markers.txt` | always |
| 2 | `NAME.MRK` in the same folder as the cut file, or in the parent folder | same folder: always; parent: pieces must match |

| | | |
|---|---|---|
| 3 | The marker open in Optitex Marker (window title read every 2 s, remembered 30 min): a path, or a name looked up in the marker folders | pieces must match |
| 4 | `NAME.MRK` in the marker folders and watched folders (newest/closest in time first) | pieces must match |
| 5 | Any `.MRK` in those folders whose pieces lie at the same places as in the cut file (up to 400 candidates, closest in time first) | GBR: ≥70 % of the file's pieces and ≥50 % of the marker's pieces at the same place (±2 mm); HPG: ≥90 % of the marker's pieces |

"Pieces must match" for a same-name marker means: at least half of its pieces lie at the same place as pieces of the cut file (±2 mm), or at least 90 % have the same sizes. This protects against an old marker that happens to have the same name.

**Pattern (.PDS):** first next to the cut file; otherwise the path stored in the marker; if that path does not exist (other computer, other drive letter) the same file name is looked up next to the marker and in the marker folders. The PDS gives the expected number of cut notches per piece ( `<T_NOTCH><CUT><COUNT>` ); without it the total from the marker is used.

## 4.1 Marker folders and the index

- "Marker folders" (Settings) can be a whole server folder, including network paths. They are walked **in the background** (at start, when the folders change, every 30 minutes, or with the button "Обходи папките отново" = "Walk the folders again"). Only `.MRK` / `.PDS` names are kept; other files are not opened. Safety stop at 5,000,000 entries. A cut file never waits for the walk. Files processed before the first walk ended are re-checked after it.
- New or changed markers are added at once from the folder watcher.
- Piece outlines of markers already read are kept in `markers-cache.txt` (by modification time), so each marker is read over the network only once.
- Watched folders are also searched, but a drive root is never walked as a watched folder (too slow); it can be added as a marker folder on purpose.
- Measured on a local disk: 50,000 files (10,000 markers) walked in 0.2 s; content search 10–30 ms. Over the network it is dominated by round trips per folder (estimate: seconds to minutes for 10,000+ folders; **not measured**).

## 4.2 Marker still being saved

- A marker or pattern is read only after it has not changed for `marker_quiet` seconds (default **3**, setting 0–120). Time of the file is used when it is clearly older; otherwise size and time are watched. Up to 60 s + the quiet time are waited; then the cut file waits and is re-checked after the next change.
- A cut file processed without a marker waits (up to 24 h) and is processed again automatically when a `.MRK` appears in a searched folder.
- On Windows all files are opened with read+write+delete sharing ( `readShared` ), so Marker can overwrite, rename or delete them while GBR Fix reads.

**Read-only for markers.** The program never writes, moves or deletes `.MRK` / `.PDS` . It writes only: the corrected cut files, and its own files in `%LOCALAPPDATA%\GbrFix` (see section 7). One exception by layout: if a folder is watched *and* holds cut files, the corrected files are written there (next to the originals).

### 4.3 The marker open in Optitex Marker

Every 2 s the program lists visible windows whose title contains "Optitex Mark", or whose program is named `*mark*`, or is in an Optitex folder (browsers and Explorer are excluded). Optitex 26 shows `NAME - Optitex Mark 26` (observed). A path in the title is used as is; a name is looked up in the marker folders. The result is shown in Settings ("Отворено в Optitex Marker: …").

## 5. The window and the tray

- **Tray icon** (next to the clock): click = open the window; right button = Open / Open folder of the last files / Pause / Exit. Closing the window does **not** stop the program. A new version started by double-click asks the running one to quit and replaces it.
- **Window:** an Edge (or Chrome) window in *--app* mode (no tabs, no address bar) showing the page served by the program on `http://127.0.0.1:8765`; own browser profile in `%LOCALAPPDATA%\GbrFix\window`. Without Edge/Chrome the default browser is used.
- **Dialogs** for folders/files are the standard Windows dialogs, owned by the GBR Fix window so they open in front of it (2.2).
- **Notifications:** one file = "corrected file + name + folder"; many files within about 20 s = one grouped notification. Click opens the folder with the file selected. Warnings are separate.
- **Start with Windows:** turned on at first start (registry `HKCU\…\Run`, with `-background`); can be switched off in Settings.

| Tab (Bulgarian) | Meaning |
|---|---|
| Последни файлове | Recent files: time, status (Коригиран = corrected, Провери = check, Грешка = error, Инфо = info), file path, outputs, the marker used and how it was found, the "restored from" line, the message. Buttons stacked per entry: *Преди / след* (before/after preview), *Преглед с върнати нотчове* (preview with restored notches), *Покажи в папката* (show in folder), *Посочи / Друга .MRK...* (choose / other .MRK). |
| Преглед | Preview of GBR, HPG, MRK and PDS (drag and drop or "Open file"): cuts white, notches green (points at start and end), red = phantom cut, grey = travel, orange ring = contour start. Coordinates on hover; two synchronised views; MRK/PDS show every notch with coordinates. |
| Ръчно конвертиране | Manual conversion: drop GBR/HPG (with a MRK) and download the result. |
| Настройки | Settings, saved automatically ("Запазено"); see section 6. |

Opening the program with a file argument (Open with → GbrFix.exe) shows that file in the preview. Command line: `GbrFix.exe -background` (start hidden), `GbrFix.exe -convert in.gbr [out.gbr]` (convert one GBR, print the report).

## 6. Settings reference (`%LOCALAPPDATA%\GbrFix\gbrfix.ini`)

| Key | Default | Meaning |
|---|---|---|
| `watch` | (none) | Folders watched with sub-folders, separated by `;`. Nothing is converted until one is chosen. A whole drive is allowed but not recommended. |
| `marker_dirs` | (none) | Folders where Marker saves `.MRK` / `.PDS` (whole server folders allowed). |
| `exclude` | `\Windows\` `\$Recycle.Bin\` `\System Volume Information\` `\AppData\Local\Temp\` `\AppData\Local\Microsoft\` `\GbrFix\` | Paths containing these parts are skipped (watching and marker search). |
| `suffix` | `_fixed` | Added to the name of the corrected file. |
| `out_mode` | `suffix` | `suffix`: next to the original; `folder`: in `FOLDER_fixed\` with the original name. |
| `neardup` | 3 | Near-duplicate notch tolerance (0.1 mm units); 0 = off. |
| `shape_v_to_i`, `shape_box_to_i`, `shape_u_to_i` | 0 | Turn V / box / U shape notches into I notches. (T notches are always converted.) |
| `close_broken_u` | 1 | Close a U notch that does not return to the edge. |
| `recover_broken_i` | 0 | Repair I notches damaged by old converters. |
| `check_missing` | 1 | Compare with the marker/pattern and warn. |
| `restore_mode` | `separate` | `off` / `same` / `separate` (section 3.2). Applies to GBR; HPG restoring uses the same on/off. |
| `segment_tol` | 400 | Squared distance (0.1 mm units) for placing a restored notch on a GBR contour segment. |
| `marker_quiet` | 3 | Seconds a marker must be unchanged before it is read (0–120). |
| `notify` | 1 | Notifications. |
| `port` | 8765 | Local window port (restart needed). |

| | | |
|---|---|---|
| `report_next_to_file` | 0 | Also write a `.txt` report next to the corrected file. |

## 7. Files the program writes

| Where | What |
|---|---|
| Next to the original (or in `FOLDER_fixed`) | `NAME_fixed.GBR\|HPG`, `NAME_fixed_restored.GBR`, optionally `.txt` report. |
| `%LOCALAPPDATA%\GbrFix\` | `gbrfix.ini` (settings), `log.txt`, `reports\` (a report per corrected file), `markers.txt` (markers chosen by hand, up to 2000), `markers-cache.txt` (piece outlines of markers read), `window\` (Edge profile of the program window). |
| Registry | `HKCU\Software\Microsoft\Windows\CurrentVersion\Run\GbrFix` when start with Windows is on. |

## 8. Source code map (for continuing the work)

Go 1.22+, single module `gbrfix`, no external dependencies (Win32 and COM through `syscall`). About 6,900 lines.

| File | Role |
|---|---|
| `app.go` | Config load/save, events, file watching glue, `process` / `convertFile`, output paths, HTTP handlers (`/api/status, config, show, pause, pickfolder, reindex, pickmrk, analyze, convert, reveal, quit`), `main` (single instance, version replacement, CLI flags). |
| `convert.go`, `shapes.go` | GBR notch repair (port of converter v16), shape notches (V/box/U), header check, reports (Bulgarian). |
| `sim.go` | ViewCut simulation: classifies moves (cut, travel, notch, phantom); also the preview data. Dispatches HP-GL to `hpgSim`. |
| `mrk.go` | Parser of the SVG drawing inside `.MRK` / `.PDS` (pieces, seams, notches, drills; MRK vs PDS), coordinate report, `RestoreMissing` for GBR. |
| `missing.go` | Notch-count check against marker/pattern (PDS counts × copies), GBR piece boxes. |
| `markers.go` | `FindMarker`, background index with cache, quiet-read logic, remembered choices, waiting list and retry. |
| `markerhints.go`, `markerapp_windows.go` | Marker window titles: parsing and Windows window enumeration. |
| `hpg.go`, `hpgapp.go` | HP-GL parser, notch finder, fit, `RestoreHPG`, preview; HPG file handling in the app. |
| `notify.go` | Batching of notifications. |
| `tray_windows.go`, `platform_windows.go`, `dialog_windows.go` | Win32 tray and menu, balloon notifications, window start, directory watchers, autostart, folder/file dialogs (IFileOpenDialog via COM, PowerShell fallback), shared file reading. |
| `platform_other.go` | Stubs so the program builds and its tests run on Linux/macOS. |

| | |
|---|---|
| `web/index.html` | The whole window (HTML, CSS, JavaScript, Bulgarian texts); embedded in the exe. `assets/` icons; `rsrc_windows_amd64.syso` icon resource. |

### Build, test, run

```
cd GbrFix/source
go vet ./...  &&  GOOS=windows go vet ./...
go test ./...                      # unit tests (Linux is enough)
GOOS=windows GOARCH=amd64 go build -trimpath -ldflags "-H windowsgui -s -w" -o
../GbrFix.exe .

# window without Windows (Linux): go build -o gbrfix . && ./gbrfix -background
#   then open http://127.0.0.1:8765 (state is kept in /tmp/GbrFix)
```

The HTTP server accepts only `127.0.0.1:PORT` / `localhost:PORT` hosts (DNS-rebinding protection) and requires the header `X-GbrFix: 1` on POST (CSRF protection); the viewer reads only `.gbr/.hpg/.mrk/.pds` paths.

### Tests (19)

Marker lookup (same folder, parent, marker folder, content, renamed file, block-cut pieces, wrong same-name marker, Optitex window title, window recognition), remembered choice, waiting file re-processed, background walk and cache, marker not read while being saved, PDS path fallback, drive roots not walked, HPG restore (form, position, idempotence, no-fit, file type), GBR start-of-contour notch. Tests use **synthetic** files; the customer sample files (not in the repository) were used for the manual checks described in section 10.

## 9. Version history

| Ver. | Date | Change |
|---|---|---|
| 2.0 | received | Delivered as `GbrFix_2.0.zip` (exe + source). Watches folders, converts GBR, T→I with M19, shapes, duplicates, restore from a `.MRK` *next to the GBR*, own checks, preview, tray, settings, notifications. |
| 2.1 | 2 Oct | Marker/pattern found outside the GBR folder: marker folders, lookup order, same-name check by pieces, content match, chosen-by-hand button, GBR waits for its marker (24 h), PDS path fallback. |
| 2.2 | 2 Oct | Background index for whole server folders, box cache, index state in Settings; dialogs open in front of the window (IFileOpenDialog); Explorer opens in front. |
| 2.3 | 2 Oct | Marker open in Optitex found from the window title; "restored from" line with full path; buttons stacked per entry. |
| 2.4 | 2 Oct | Recognises the real Optitex 26 window ("Optitex Mark 26"). |
| 2.5 | 2 Oct | Markers read only when unchanged and opened with full sharing (never blocks Marker). |
| 2.6 | 2 Oct | Wait before reading a marker is a setting, default 3 s. |
| 2.7 | 12 Oct | HP-GL (.HPG) support: missing notches restored; start-of-contour GBR notch no longer doubled; looser marker recognition for block-cut pieces; HPG preview. |

# 10. What is verified and what is not

## Verified (on Linux, with the sample files)

- GBR conversion output equalled the HTML converter v16 byte for byte on the first sample (stated in the 2.0 hand-over); phantom-cut count after conversion is 0 on the CH-CHINO sample.
- All 28 missing HPG notches of the CH-CHINO sample were located and inserted; visual check of six of them (shape, side, place); second pass adds nothing; only lines are added to the file.
- Marker lookup, background index, quiet-read and wait/retry logic through unit tests; the window and settings through a headless browser.
- The Windows build compiles and passes `go vet`.

## Not verified – check first on Windows

1. **Nothing has been run on Windows:** tray, balloons, directory watching, autostart, the standard file/folder dialogs (COM code), window start in Edge, the "in front" behaviour, shared file reading.
2. **No output has been cut** on the machine – neither GBR nor HPG. Cut a trial marker and check at least one restored notch (for the sample: piece grub at X316 Y1470, piece gr-nasr at X12420 Y7687).
3. **Optitex window title:** only "NAME – Optitex Mark 26" has been seen (screenshot); whether it contains a full path, and the exe name, are not known.
4. **Network speed** of the marker walk on the real server (10,000+ folders) – only a local measurement exists.
5. **Other Optitex builds / HP-GL settings:** only one HPG sample (Metric 1/40, T notches). Other resolutions are handled by design but untested. The exact Optitex build of the samples was not recorded.
6. The earlier claim "26.1 drops one notch per piece type in GBR" comes from the hand-over; it was not re-checked here and may be the same defect as in the HPG.

# 11. Known limits and ideas for next steps

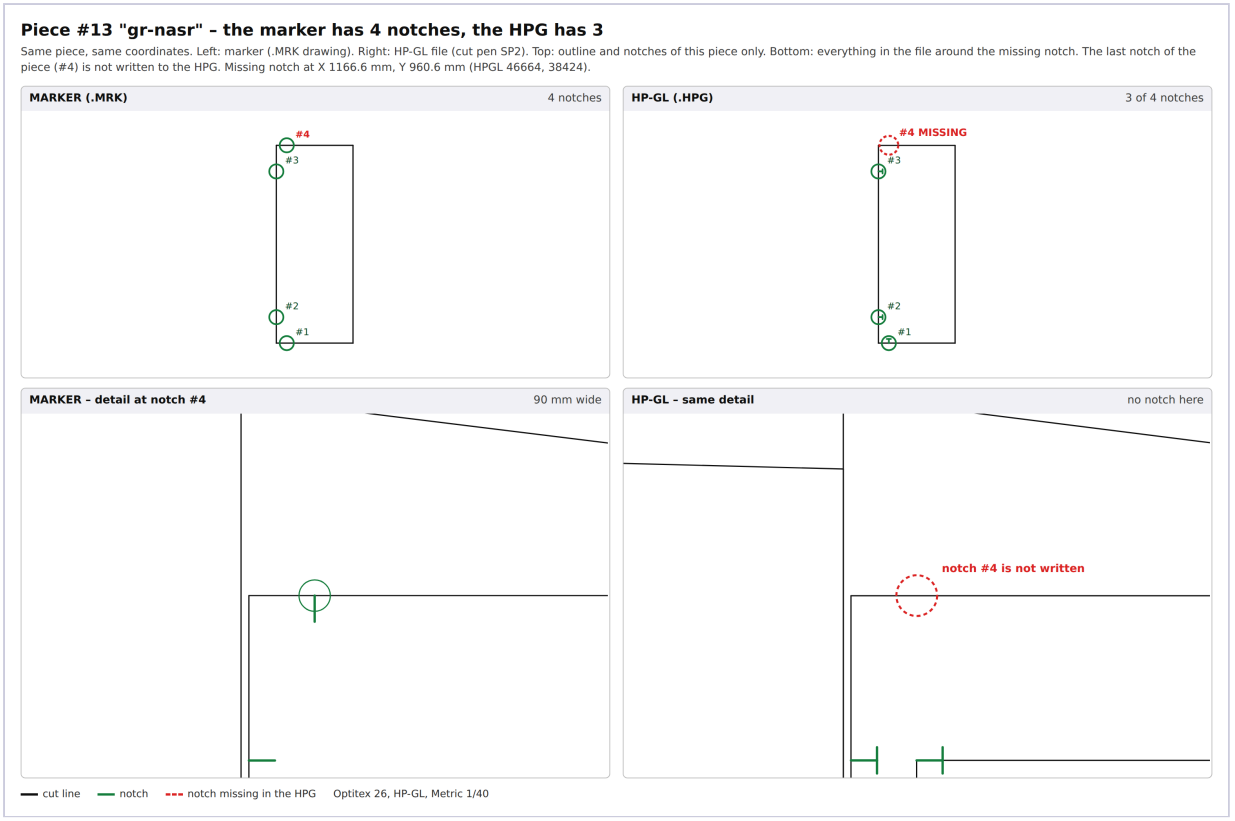
- **Pieces cut as a block in the GBR** (rectangular outline around several pieces): their notches cannot be placed in the GBR because there is no contour for them; the report says so (38 notches in the sample).
- HPG: only T and I notch forms; text/drill pens are not touched; no check against the PDS (only the marker).
- GBR restore adds only I notches with `M19` (the form the cutter handles).
- The program is not code-signed: Windows shows "Windows protected your PC" (More info → Run anyway). An installer or signing would remove this.
- No log rotation (`log.txt` grows).
- Possible next steps: run the Windows checks above and fix what appears; ask Optitex for a fixed build (then the restore step becomes a safety net); read the marker name from the GBR header when present; other cutter formats if needed; Bulgarian/English switch for the window.

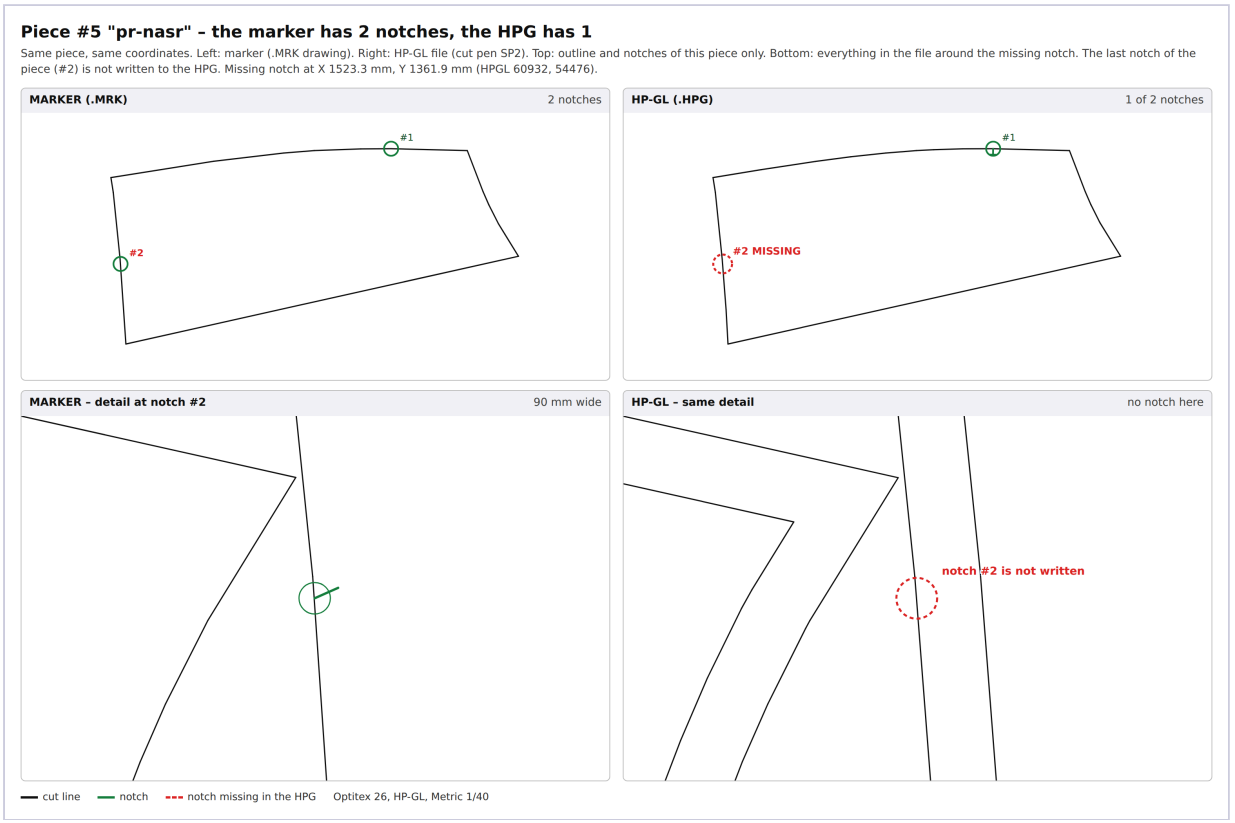
## 12. Starting again in another place (checklist)

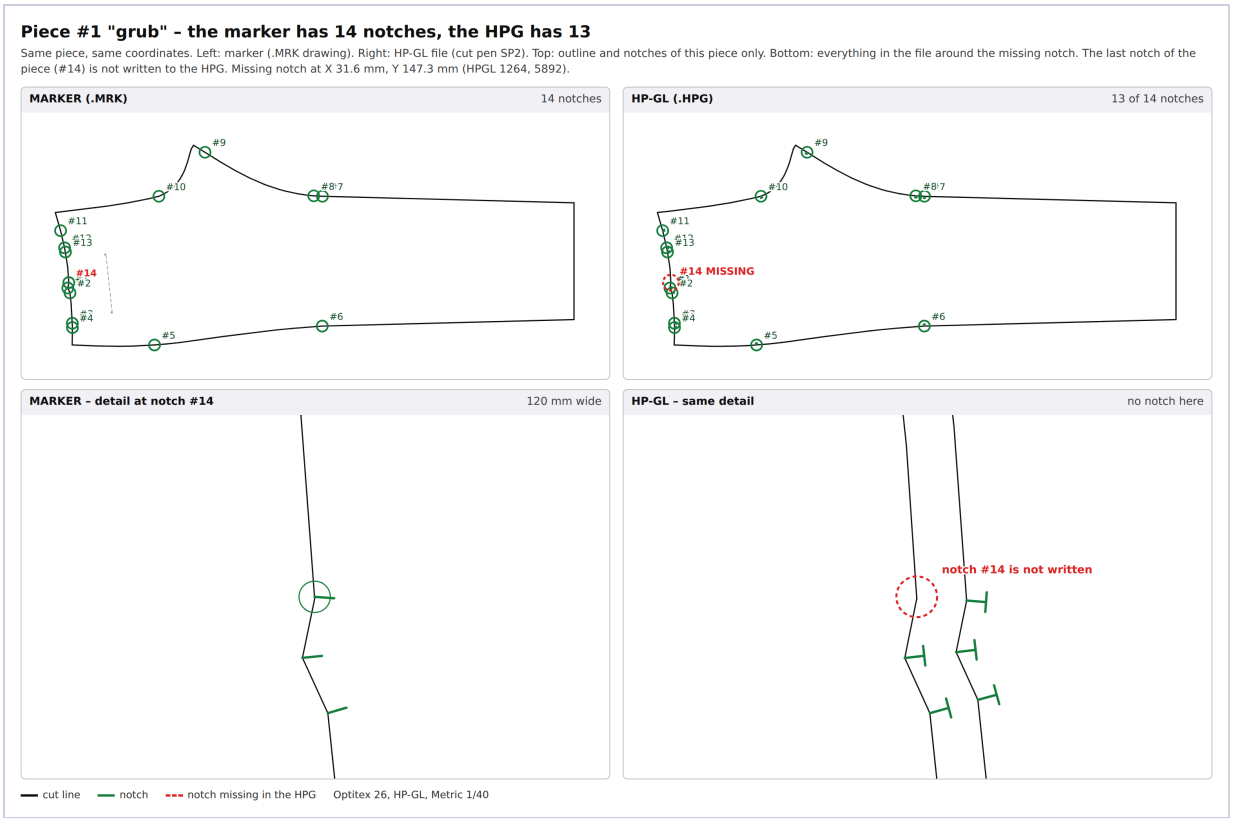
1. Clone `nggkroze/teokroze`, branch `claude/gbr-fix-notch-correction-uub3kq`; read `GbrFix/README.txt` (user guide, Bulgarian) and this handbook.
2. Run the build/test commands of section 8. Expect all tests to pass.
3. Get the sample set again from the owner: the `.MRK`, `.PDS`, `.HPG` and `.GBR` of *CH-CHINO-Atatex-PR-LICE-32-44* (not in the repository, customer data).
4. On a Windows PC: copy `GbrFix.exe` to a fixed folder, start it, choose the watched folder and the marker folder (section 6), export a trial GBR/HPG from Marker and follow the "Recent files" entry.
5. Work through section 10 "Not verified" and record the results.

# Appendix A – the HPG defect, three pieces (marker vs file)

Each picture shows the same piece in the marker (left) and in the HPG (right); the bottom row is the detail at the missing notch, with everything in the file around it. Notches are green, the missing one is red.






# Appendix B – notch forms at a glance

| Form | Text in the file | Cutter |
|---|---|---|
| GBR, old (15–21) and corrected by GBR Fix | `A*M19*B*M15*A*M14` inside the contour: edge point A, notch to B (3 mm), knife up, back to A, knife down | cuts correctly |
| GBR, Optitex 25/26.0 (front) | `A*M14*M19*B*M15` … in front of the contour | wrong (red lines) |
| GBR, 26.0.4.2 / 26.1 T as cut lines | slit plus 3 mm crossbar as ordinary cuts, no `M19` | wrong; converted to I with M19 |
| GBR, V / box / U shapes | part of the contour | cut as a shape; U leaves a sliver |
| HPG, T notch (Optitex 26) | `PD E; PD I; PU C1; PD C2; PU E` (1/40 mm units) | last notch of each piece missing |

GBR Fix – project handbook, state after version 2.7. Prepared from the work done in the development sessions of 2–13 October 2026.